Aula 09 - Randomized computation

Hi guys

Ok, I debated myself about the options of what to teach next.

Last year, I proved Hastad's lemma, talked about constant-depth circuits, and eventually proved Razborov's result that monotoneNP $\neq$ monotoneP.

These proofs are quite intricate, and nice, but I am not sure if there is much profit to be had, for the students especially, in learning them.

So I have decided to make them optional topics instead: for the presentations.

Instead, I will try to focus on more conceptual results in complexity theory.

The plan, then, is:

- We will learn about randomized algorithms (that's today)
- We will learn about hardness versus randomness.
- We will learn about cryptography.
- We will learn about the natural proofs barrier.

This is 4 topics, for 6 classes. Hopefully we can do them all. The last three topics are beautiful results, all of them, and they all require us to understand a bit more about randomness and randomized computation.

Randomized Turing Machines

Definition. A *randomized Turing machine* is just a non-deterministic Turing machine, formally, but with a conceptual difference. We think of a non-deterministic Turing machine as "guessing the right answer" and then checking it. But we think of a randomized Turing machine as operating randomly.

The machine has two transition functions, just like a NDTM, but we think of the computation as follows: at every time step, the machine tosses an unbiased coin, and chooses to apply δ_0 or δ_1 depending on the outcome of the coin.

One way of thinking about such algorithms is that they have a subroutine `randombit()` which when invoked returns 0 or 1 each with probability $1/2$.

Thinking of it like this, then the *computation* of a randomized Turing machine, i.e., the sequence of configurations of its computation, is not a fixed value depending only on the input, but rather a random variable. One can then talk about the probability that the machine outputs 0 or 1, etc.

Definition. The time taken by a probabilistic Turing machine on input x is the maximum, over all possible choices of the random coins $r \in \{0,1\}^*$, of the time taken by the computation of M on input x with randomness r.

Notation. We usually write $M(x, r)$ to denote the computation of M on input x if the outcome of the tossed coins is r. We will see later that this makes a lot of sense.

Definition. We say that a Probabilistic TM (PTM) computes a function $f : \{0,1\}^* \to \{0,1\}$ ***with bounded error*** if

$$\Pr_r[M(x, r) = f(x)] \geq 2/3.$$

We say that M computes f with *one-sided error* if

$$f(x) = 0 \implies M(x) = 0 \text{ always}$$

$$f(x) = 1 \implies M(x) = 1 \text{ with probability } \geq 2/3$$

> why 2/3? good question, more on that later.)

We say that M computes f with *zero error* if $M(x) \in \{0, 1, ?\}$ always ("0, 1, don't know"), and

$$\Pr_r[M(x, r) = f(x)] \geq 2/3.$$

(so M always answers 0, 1, or don't know, but the probability of saying don't know is $\leq 1/3$).

Definition.

- $\mathsf{BPTIME}(t)$ is the class of functions that can be computed by PTMs with bounded error, and which on every input x run in time $t(|x|)$ (or less).
- $\mathsf{RPTIME}(t)$ is the class ..... with one-sided error ....
- $\mathsf{ZPTIME}(t)$ .... zero-error ....

We then have the polynomial-time classes BPP, RP and ZPP.

Ok, so this is the analogous concept to polynomial time, but with randomness. You can see we have three acceptance criteria: bounded error is the easiest criterion to satisfy, then one-sided error, then zero error:

Homework. Show that $\mathsf{ZPP} \subseteq \mathsf{RP} \subseteq \mathsf{BPP}$.

Algorithmic examples

Ok, so we extended our algorithms to allow them to toss coins. And we relaxed the definition of when an algorithm is correct, to say "an randomized algorithm is correct if it is correct most of the time, i.e. with reasonably high probability."

Apriori it is not at all clear why randomness should help. But it turns out that it does, quite a lot. Let us see some examples.

Median in $\mathsf{ZPTIME}(wn)$

Let us show that there is a simple randomized algorithm for finding the median which appears best than the simple deterministic algorithm for finding the median.

Consider the function $f = \mathsf{FindKthElement}$, which when given as input a number $k \in [n]$ and a list $L = (a_1, \ldots, a_n) \in [2^w]^n$ of n w-bit numbers, should output the k-th smallest element in the list.

Ok?

Give me a simple deterministic algorithm to compute $f(k, L)$. Sorting!

To sort L takes how much time? $O(wn \log n)$

So we sort the entire list, this takes $O(wn \log n)$ time, and then we traverse the sorted list from left to right until we reach the k-th position, which we copy to the output tape. This is an extra $O(wn)$ time so it doesn't matter much.

However, here's a very simple *randomized* algorithm for computing f:

```
findkthsmallest(k, L):
    Choose a random position i in {1, ..., |L|}
    Let a = L[i]
    Traverse the list L and count the number of elements <= a
    Let m be this number
    if by luck m = k, then output a
    if m > k, then let L' be the list containing all elements of L that are < a
       output findkthsmallest(k, L')
    if m < k, then let L' be the list containing all elements of L that are > a
       output findkthsmallest(k-m, L')
```

Claim. This algorithm always outputs the correct number (check this.) And it runs in expected time $O(wn)$

Proof. Let $T(w, n)$ be the maximum expected time taken by the above algorithm on lists of length n with w-bit numbers. (maximum over lists, expected over random choices of the algorithm.)

Let us prove that $T(w, n) \leq 10cwn$ for some constant c. This is obvious if n itself is some constant (by choosing c large enough). So let us assume it's true up to lists of size at most $n - 1$ and prove it for n.

Notice that:

$$T(w,n) \leq \underbrace{c \cdot w \cdot n}_{*} + \frac{1}{n}\left(\underbrace{\sum_{j>k} T(w,j)}_{\text{case 1}} + \underbrace{\sum_{j<k} T(w,n-j)}_{\text{case 2}}\right)$$
$$=^{(Ind.Hyp.)}$$

- -- the time taken to compute a and m, plus output the result if we are lucky and $k = m$

... etc finish (HW)

Example 2 - Checking if a number is prime

Since 2009 (?) we have known a deterministic algorithm for deciding if a given n-bit natural number is prime, that runs in $poly(n)$ time. This was a tremendous achievement after many decades of work.

But a randomized algorithm that runs in nearly linear time had been known for decades. It is still the algorithm that is used in practice (the deterministic algo runs in n^7 time or some ridiculous number like that, so it is impractical).

Also, one of your colleagues last year did a presentation on the deterministic algo. It took about 6 hours to give us a vague idea of what was happening. Instead, the randomized algo is a one-page explanation:

We now sketch an algorithm showing that PRIMES is in **BPP** (and in fact in **coRP**; see Section 7.3). For every number N, and $A \in [N-1]$, define

$$QR_N(A) = \begin{cases} 0 & gcd(A,N) \neq 1 \\ +1 & A \text{ is a } \textit{quadratic residue} \text{ modulo } N \\ & \text{That is, } A = B^2 (\mathrm{mod} N) \text{ for some } B \text{ with } gcd(B,N) = 1 \\ -1 & \text{Otherwise} \end{cases}$$

We use the following facts, all of which can be proven using elementary number theory (e.g., see [Sho05]):

- For every odd prime N and $A \in [N-1]$, $QR_N(A) = A^{(N-1)/2} \pmod N$.
- For every odd N, A define the *Jacobi symbol* $(\frac{N}{A})$ as $\prod_{i=1}^{k} QR_{P_i}(A)$ where $P_1, \ldots, P_k$ are all the (not necessarily distinct) prime factors of N (i.e., $N = \prod_{i=1}^{k} P_i$). Then, the Jacobi symbol is computable in time $O(\log A \cdot \log N)$.
- For every odd composite N, among all $A \in [N-1]$ such that $gcd(N,A)=1$, at most *half* of the A's satisfy $(\frac{N}{A}) = A^{(N-1)/2} \pmod N$.

Together these facts imply a simple algorithm for testing primality of N (which we can assume without loss of generality is odd). *Choose a random* $1 \leq A < N$. *If* $gcd(N,A) > 1$ *or* $(\frac{N}{A}) \neq A^{(N-1)/2} \pmod N$ *then output "composite"; otherwise, output "prime."* This algorithm will always output "prime" if N is prime, but if N is composite, it will output "composite" with probability at least $1/2$. Of course, this probability can be amplified by repeating the test a constant number of times.

Let's not go over it (we won't get much out of it without also learning the required number theory), but you can see that it is not so complicated.

Example 3 - Polynomial identity testing

Input. We are given an arithmetic circuit C of size s with n variables:

- An arithmetic circuit is like a circuit, but where the wires don't carry bits 0/1, but carry integers. The allowed gates of the circuit are $+$, $\times$, input gates x_i, and integer constant gates.
- Any arithmetic circuit computes a polynomial of its input variables. (obvious?)

The polynomial identity testing is the problem of deciding, when given such an arithmetic circuit, whether it computes the zero polynomial. I.e. whether $C(x) = 0$ for all $x \in \mathbb{Z}^n$.

i.e.:

- Input: An arithmetic circuit of size s with n variables.
- Output: Is it true that $C(x) = 0 \forall x$?

It's not obvious how to do this.

A hint is that a polynomial has a lot of redundant information. E.g. a univariate polynomial of degree n has at most n zeros. Likewise there exists a similar theorem for multivariate polynomials:

Schwartz-Zippel Lemma. Let $S \subseteq \mathbb{Z}$ be ANY FINITE SET. If $p(x_1, \ldots, x_n)$ is a **non-zero** polynomial of maximum total degree d, and we choose $a_1, \ldots, a_n$ uniformly and independently from S, then

$$\Pr[p(a_1, \ldots, a_n) \neq 0] \geq 1 - \frac{d}{S}$$

So now how do we test if $p = 0$? proposals?

Suggestion 1: Take ANY set S of our choosing, e.g., $S = \{1, 2, 3, \ldots, 4d\}$. Sample $a_1, \ldots, a_n$ uniformly at random from S. Then compute $C(a_1, \ldots, a_n)$. If it outputs 0, we say $C = 0$, otherwise we say $C \neq 0$.

By the schwarz-zippel lemma, this algorithm outputs the correct answer with probability $\geq 3/4$.

why doesn't this work? (it's kinda non-obvious)

This doesn't work because arithmetic circuits of size s can compute numbers as large as 2^{2^s}.

The algorithm is then:

Choose a random number $k \in [2^{2n}]$, and compute $C(a_1, \ldots, a_n) \mod k$. Accept (output "$C = 0$") iff this number is zero.

To prove that this works:

- If $C = 0$ then we always have $C(a) \mod k = 0$.
- If $C \neq 0$, say that $y = C(a) \neq 0$. Then one can show that the number y is at most $2^{O(s2^s)}$ (homework), and such a number can have at most $\log y = O(s2^s)$ factors. So when choosing a random k, it is likely that we did *not* choose a factor of y, in which case $y \mod k \neq 0$

OPEN PROBLEM. Does there exist a deterministic algorithm for polynomial identity testing?

IF YOU SHOW THIS THEN IT MIRACULOUSLY FOLLOWS THAT $NEXP \neq P/poly$. That's really non obvious (why would an upper-bound imply a lower-bound?!) (also tragic: we do not have a proof that $NEXP \neq P/poly$)

(another way of thinking about this is: it is probably quite hard to prove that poly-identity-testing is in P, because to show this we need to prove lower-bounds along the way :-)

(however, we will show that if $E \not\subseteq SIZE(2^{\varepsilon n})$ then BPP $=$ P and hence PIT is in P! this is the hardness vs randomness)

Robustness of the definition

Okay, now we have seen some examples, that show that we can really make use of randomness to make our algorithms faster and/or simpler.

Some things in our definitions seem arbitrary. E.g. why 2/3?

- It turns out that the complexity classes BPP, ZPP and RP are robust to changes in the definition.
- E.g. if we have an algorithm that outputs $f(x) \in \{0, 1\}$ correctly with probability 2/3, then we can run the algorithm $O(n)$ times (with fresh randomness), and take the majority of the answer. It turns out (HW) that the answer will then be correct with probability $1 - 2^{-n}$, i.e., inverse-exponentially close to 1.

- Also, if we have an algorithm that outputs $f(x)$ with probability slightly larger than $1/2$, say $1/2 + \frac{1}{poly(n)}$, then we can again repeat the algo and take majority answer and this will boost the probability of success. (this doesn't work if the answer is correct only with $1/2 + 1/2^n$ probability, say. Why? Homework)
- Notice that if we have one-sided error (RP or ZPP), then it is enough that the algorithm is correct with probability $1/poly(n)$. Because we can then repeat the algorithm $poly(n)$ times: if the output was 0, then all the repetitions output 0; if the output should be 1, then after poly(n) repetitions we are very likely to have seen a 1. (again this will be homework, with hints etc)

Basic relationships among the discussed classes

$$\mathsf{ZPP} = \mathsf{RP} \cap \mathsf{coRP}$$

proof. Let $f : \{0,1\}^* \to \{0,1\}$. Suppose that $f \in \mathsf{ZPP}$. I.e. we have a polytime randomized algorithm A that outputs $0, 1$ or $?$, which is never wrong, and outputs 0 or 1 with probability $\geq 3/4$.

Then we run $A(x)$, and if it outputs 1 or ? we also output 1, if $A(x)$ outputs 0, we output 0. Because A is never wrong, we are never wrong when we output 0, though we might be if we output 1. This shows that f is in RP also.

The proof that $f \in \mathsf{coRP}$ is the same, but we output 0 if $A(x) =?$.

Converse: HW.

$$\mathsf{RP}, \mathsf{coRP} \subseteq \mathsf{BPP}$$

(hence $\mathsf{ZPP} \subseteq \mathsf{BPP}$ also)

Notice the conditions for an RP algorithm to compute f with one-sided error. They imply that the very same algorithm also computes f with bounded error.

$\mathsf{RP} \subseteq \mathsf{NP}$, $\mathsf{coRP} \subseteq \mathsf{coNP}$

Notice the conditions for an RP algorithm to compute f with one sided error. If $f(x) = 0$, then every choice of randomness makes the machine output 0. If $f(x) = 1$, then at least 3/4 of the random choices make the machine output 1. In particular *there exists* a choice that makes it output 1. so these conditions imply that the same algorithm is a verifier for f, where the certificate is the randomness.

$\mathsf{BPP} \subseteq \Sigma_2$ -> is not obvious, it will be part of the extra stuff you learn by yourself and may appear in the test.